

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****CO1 ASSESSMENT TOOL 1 – Case Study Analysis**

Particulars	Details
Course Code	CSA10 / CSA1023
Course Title	Software Engineering
CO Assessed	CO1
Assessment	Assessment Tool 1 – Case Study Analysis
Weightage	20%
Total Marks	25
CO1 Statement	Cloud Cost Anomaly Detection and FinOps Automation Platform
Student Name	N Reddy varaprasad reddy
Reg. No.	192311045
Date	07/10/2026
Duration	60 minutes

Annexure A – Case Study Analysis**Instructions to Students:**

Each student will be allotted one problem statement. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. Understand the Problem Statement – Explain the problem in your own words, identify the objectives, and state the scope.
2. Identify Key Stakeholders and Challenges – Identify stakeholders, roles and responsibilities, and major challenges.
3. Analyze the Current Approach – Describe the existing process, strengths, limitations, and gaps.
4. Propose a Suitable Solution Using Software Engineering Principles – Design the solution, modules, requirements, and engineering principles.
5. Justify the Chosen Methodology/Framework – Select and justify an appropriate SDLC model or framework.
6. Discuss Expected Outcomes and Limitations – Benefits, challenge resolution, risks, limitations, and future enhancements.

Code of Conduct

I N Reddy varaprasad (192311045) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N Reddy

Table of Contents

- 1. Understanding the Problem Statement
 - 1.1 Problem Explained in Own Words
 - 1.2 Objectives of the Proposed System
 - 1.3 Scope of the Problem
- 2. Key Stakeholders and Challenges
 - 2.1 Stakeholders
 - 2.2 Roles and Responsibilities
 - 2.3 Major Challenges and Issues
- 3. Analysis of the Current Approach
 - 3.1 Existing System / Current Process
 - 3.2 Strengths and Limitations
 - 3.3 Problems and Gaps to be Addressed
- 4. Proposed Solution Using Software Engineering Principles
 - 4.1 Overview and Architecture
 - 4.2 Major Functional Modules
 - 4.3 Functional and Non-Functional Requirements
 - 4.4 Software Engineering Principles Incorporated
- 5. Justification of the Chosen Methodology
 - 5.1 Selected Framework
 - 5.2 Why It Is Suitable
 - 5.3 Support for Development and Maintenance
- 6. Expected Outcomes and Limitations
 - 6.1 Expected Benefits
 - 6.2 How Challenges Are Addressed
 - 6.3 Risks, Implementation Challenges and Limitations
 - 6.4 Future Enhancements
- 7. Conclusion
- 8. References

1. Understanding the Problem Statement

1.1 Problem Explained in Own Words

Cloud platforms are widely used to run applications, databases, containers, analytics workloads and DevOps pipelines. Cloud resources can experience unexpected cost increases because of resource leaks, incorrect configurations, excessive workloads, over-provisioning, idle resources, or inefficient deployment practices. Manual analysis often identifies these problems only after a billing period has ended, when the organization has already incurred unnecessary expenditure.

The proposed problem is to design and develop a Cloud Cost Anomaly Detection and FinOps Automation Platform that continuously monitors cloud resource consumption and expenditure, identifies unusual cost patterns, and provides actionable recommendations. The platform should connect cost observations with the resources, teams and deployments responsible for them.

The system should combine monitoring, anomaly detection, dashboards, alerts, infrastructure-as-code checks, container and CI/CD information, policy validation, and controlled optimization actions. Potentially risky changes should require approval, while low-risk actions can be automated according to predefined policies. Every cost-related change should be recorded in an audit trail.

1.2 Objectives of the Proposed System

- Monitor cloud resource consumption and associated costs continuously.
- Detect unusual cost patterns earlier than end-of-month billing analysis.
- Identify inefficient resource configurations such as idle, oversized or under-utilized resources.
- Provide optimization recommendations based on usage, cost and configuration information.
- Support controlled automated remediation with approval workflows and policy guardrails.
- Integrate infrastructure-as-code so cost policies can be checked before deployment.
- Integrate CI/CD pipelines to detect potentially expensive configuration changes before release.
- Provide dashboards, alerts and reports for engineers, FinOps teams and management.
- Maintain audit trails for recommendations, approvals, automated actions and cost-related changes.
- Improve resource utilization, operational visibility and sustainable DevOps practices.

1.3 Scope of the Problem

In scope	Out of scope (for this project)
Cloud cost and usage ingestion from supported cloud accounts	Replacing the cloud provider's complete billing platform
Cost anomaly detection and alerts	Direct integration with confidential accounting systems
Resource utilization and waste identification	Uncontrolled automatic deletion of production resources
Optimization recommendations	Guaranteeing exact future savings
FinOps dashboards and reports	Training large AI models from scratch
IaC and CI/CD policy checks	Full enterprise procurement and contract negotiation
Controlled remediation with approvals	Autonomous changes without governance
Container/cluster cost visibility	Physical data-center hardware optimization
Audit logs and change history	Legal/tax certification or financial audit sign-off

2. Key Stakeholders and Challenges

2.1 Stakeholders

A stakeholder is any person, team or organization that uses, operates, manages, is affected by, or has an interest in the system.

- Cloud application owners and development teams
- FinOps / cloud cost management team
- DevOps, SRE and platform engineers
- System administrators and cloud administrators
- Finance and management teams
- Security and compliance teams
- Software developers and testers
- CI/CD and infrastructure-as-code maintainers
- Project manager / product owner
- Cloud service providers and account administrators

2.2 Roles and Responsibilities

Stakeholder	Role and responsibilities
Application / development teams	Own workloads, review recommendations, correct inefficient configurations and provide business context.
FinOps team	Monitor spending, investigate anomalies, define policies, evaluate savings and report cost trends.
DevOps / SRE	Integrate monitoring, IaC and CI/CD controls, operate the platform and implement approved actions.
Cloud administrators	Manage accounts, permissions, quotas, resource metadata and secure access.
Finance / management	Review expenditure, budgets, forecasts, savings results and business-level trends.
Security / compliance	Ensure least privilege, secure credentials, auditability and controlled automation.
Developers / testers	Build and test services, integrations, anomaly detection, dashboards and automation.
IaC / CI/CD maintainers	Apply cost-policy checks to infrastructure changes and deployment pipelines.
Product owner / project manager	Prioritize requirements, coordinate stakeholders and manage project scope.
Cloud provider	Supplies resource, billing, usage and monitoring data through supported APIs.

2.3 Major Challenges and Issues

Stakeholder	Challenges faced
Development teams	Unexpected resource costs, billing complexity, and performance/cost trade-offs.
FinOps team	Large usage volumes, delayed anomaly discovery and difficulty linking cost to technical causes.
DevOps / SRE	Balancing reliability and cost, safe automation and production risk.
Management / Finance	Limited real-time visibility, uncertain forecasts and unexplained cost spikes.
Security / Compliance	Protecting credentials, least privilege and proving automated actions are authorized.
Overall	False alarms, changing workloads, cloud complexity and noisy telemetry.

3. Analysis of the Current Approach

3.1 Existing System / Current Process

Organizations commonly use native cloud billing dashboards, budget alerts, spreadsheets, monitoring tools and periodic manual reviews. Engineers may inspect resource utilization separately from billing information, while finance teams may receive cost reports after spending has already occurred.

A typical current flow is: cloud resources are deployed → usage and billing data accumulate → reports or budget alerts are generated → an analyst investigates → inefficient resources are identified manually → recommendations are communicated → an administrator performs the change → results are checked later.

Infrastructure-as-code and CI/CD systems may exist independently, but cost policies are not always evaluated as part of the deployment workflow. An expensive configuration can therefore pass through the pipeline even when it is likely to create unnecessary cost.

3.2 Strengths and Limitations

Approach	Strengths	Limitations
Manual cost review	Human judgement and business context.	Slow, labor-intensive and reactive.
Cloud billing dashboards	Direct provider data and cost breakdowns.	Limited cross-system root-cause analysis and automation.
Budget / threshold alerts	Simple and easy to configure.	Fixed thresholds can be noisy and may alert late.
Spreadsheets / reports	Easy to share and customize.	Stale data, manual updates and weak traceability.
Standalone monitoring	Detailed operational metrics.	Often disconnected from billing and FinOps context.
Manual optimization	Can account for business requirements.	Depends on experts and can miss short-lived opportunities.

3.3 Problems and Gaps to be Addressed

- Cost anomalies may be discovered only after substantial spending.
- Billing, resource utilization, deployment and IaC information is fragmented.
- Fixed thresholds do not adapt well to normal daily, weekly or seasonal patterns.
- Alerts may lack enough evidence to identify root cause quickly.
- Recommendations may not identify the exact resource, owner, configuration or expected impact.
- Manual remediation is slow, while unrestricted automation can be risky.
- Cost checks may occur after deployment instead of preventing expensive configurations.
- Audit trails for recommendations, approvals and changes may be incomplete.
- Containerized workloads make cost allocation and efficiency harder to understand.
- Optimization must balance cost reduction with performance, reliability and governance.

4. Proposed Solution Using Software Engineering Principles

4.1 Overview and Architecture

The proposed solution is a cloud-native, API-first and modular Cloud Cost Anomaly Detection and FinOps Automation Platform. It continuously collects billing and usage data, normalizes the information, calculates baselines and anomaly scores, identifies likely root causes, and presents recommendations through dashboards and alerts.

The platform connects cloud billing/usage sources, monitoring metrics, resource metadata, infrastructure-as-code repositories, CI/CD pipelines and container platforms. A policy engine checks infrastructure changes before deployment. A recommendation engine evaluates waste and configuration opportunities. A controlled automation service executes only approved or policy-permitted actions, while every action is recorded in an audit trail.

High-level architecture: Cloud Accounts / Billing APIs → Data Ingestion → Cost Normalization & Allocation → Anomaly Detection → Root-Cause Analysis → Recommendation Engine → Dashboard / Alerts → Approval & Policy Engine → Controlled Remediation → Audit Trail

Supporting integrations: Infrastructure-as-Code repositories, CI/CD pipelines, container/Kubernetes metrics, logging systems, identity/access management and notification channels.

4.2 Major Functional Modules

Module	Function
Data Ingestion Service	Collects billing, usage, monitoring and resource metadata.
Cost Normalization & Allocation	Standardizes cost records and maps resources to owners/projects/environments.
Anomaly Detection Engine	Builds baselines and identifies unusual cost or usage patterns.
Root-Cause Analysis	Correlates anomalies with resource changes, deployments and utilization.
Recommendation Engine	Identifies idle, oversized, under-utilized or inefficient resources.
Policy Engine	Evaluates budget, tagging, resource-size and cost policies.
IaC / CI/CD Integration	Runs cost-policy checks during pull requests and deployment workflows.
FinOps Dashboard	Shows cost trends, anomalies, utilization, recommendations and savings.
Alerting Service	Sends configurable alerts to relevant teams.
Approval & Remediation Service	Routes risky actions for approval and performs controlled actions.
Audit & Reporting Service	Records recommendations, approvals, actions, users, timestamps and results.
Admin & Configuration	Manages thresholds, policies, roles, integrations and automation permissions.

4.3 Functional and Non-Functional Requirements

Functional requirements

ID	Requirement
FR1	Collect cloud billing, usage and resource metadata through secure integrations.
FR2	Normalize cost records and associate resources with owners and projects.
FR3	Calculate historical baselines and detect unusual cost or usage patterns.
FR4	Generate anomaly alerts with severity, affected resource and supporting evidence.
FR5	Correlate anomalies with deployments, configuration changes and utilization metrics.
FR6	Identify inefficient, idle, oversized or under-utilized resources.
FR7	Generate optimization recommendations with estimated savings and risk information.
FR8	Provide dashboards for cost trends, anomalies, utilization, recommendations and savings.
FR9	Run policy checks for IaC and CI/CD changes.
FR10	Provide approval workflows for controlled remediation.
FR11	Execute only authorized optimization actions and verify results.
FR12	Maintain tamper-resistant audit records for recommendations, approvals and actions.
FR13	Provide role-based access for FinOps, developers, DevOps, finance and administrators.
FR14	Provide reports and historical analysis of cost, anomalies and optimization outcomes.

Non-functional requirements

Attribute	Requirement
Security	TLS in transit, encryption at rest, RBAC, least privilege and secure secrets.
Performance	Dashboards and common anomaly queries should respond quickly; ingestion should support near-real-time or scheduled processing.
Scalability	Stateless services and horizontally scalable processing should support growing resource and telemetry volume.
Reliability / availability	Retries, health checks, durable queues and backup/recovery mechanisms.
Usability	Clear dashboards, understandable anomaly explanations and simple approval workflows.
Maintainability	Modular services, documented APIs, version control and automated tests.
Auditability	Every policy decision, recommendation, approval and remediation action must be traceable.
Interoperability	Documented interfaces for cloud, IaC, CI/CD and monitoring integrations.

4.4 Software Engineering Principles Incorporated

Principle	How it is incorporated
Modularity	Ingestion, detection, recommendation, policy, remediation and audit capabilities are separated into clear modules.
Scalability	Containerized/stateless services, asynchronous processing and horizontal scaling.
Maintainability	Clean layered design, API documentation, code review and automated testing.
Reliability	Retries, timeouts, health checks, durable queues and rollback/approval mechanisms.
Usability	Clear cost trends, anomaly severity, root-cause evidence and actionable recommendations.
Security	RBAC, least privilege, encryption, secure secrets and controlled cloud permissions.
Testability	Unit, integration, policy, pipeline and remediation tests are automated.
Observability	Logs, metrics, traces and automation outcomes are monitored.
Auditability	Audit records capture who/what/when/why for cost-related decisions.
DevOps	CI/CD automates build, testing, security checks, deployment and rollback.

Illustrative anomaly decision logic:

The anomaly engine can compare current cost against an expected baseline derived from historical usage, seasonality and workload context. A configurable policy can classify events as Normal, Warning or Critical. Thresholds should be configurable and validated against historical data rather than treated as universal values.

Controlled remediation logic:

Low-risk actions such as notifying an owner or opening an optimization ticket can be automated. Actions that change capacity, schedules or production resources should require policy authorization and, where appropriate, human approval. The platform should record the proposed action, expected savings, risk, approval, execution result and rollback information.

5. Justification of the Chosen Methodology

5.1 Selected Framework

The selected approach is Agile (Scrum) combined with DevOps practices such as continuous integration, continuous delivery, infrastructure-as-code, automated testing and continuous monitoring. Development can be divided into short sprints of one to two weeks, with each sprint producing a working and tested increment.

Sprint	Sprint Goal
Sprint 0	Requirements, architecture, repository, cloud test environment and CI setup.
Sprint 1	Data ingestion, cost normalization and basic dashboard.
Sprint 2	Anomaly detection, alerts and historical baseline analysis.
Sprint 3	Resource efficiency analysis, recommendations and root-cause correlation.
Sprint 4	IaC / CI/CD policy checks, dashboard improvements and RBAC.
Sprint 5	Approval workflow, controlled remediation and audit trail.
Sprint 6	Security, performance testing, monitoring, documentation and final validation.

5.2 Why It Is Suitable

Model	Assessment for this project
Waterfall	Poor fit because cloud costs, workload patterns and requirements can change; late validation would delay discovery of problems.
Spiral	Good risk handling, but the repeated risk-analysis overhead may be heavy for a student-scale project.
Agile / Scrum + DevOps (chosen)	Best fit because the platform can be delivered incrementally, tested continuously and adapted to stakeholder feedback.

- Evolving requirements: cloud services, pricing models and workload behavior can change.
- Technical uncertainty: anomaly detection quality and savings estimates need experimentation.
- Modular architecture: services map naturally to sprint-sized work items.
- Continuous quality: DevOps pipelines can run tests, security scans and IaC checks.
- Stakeholder involvement: FinOps, developers, DevOps and management can review each increment.
- Safe automation: approval gates reduce the risk of incorrect optimization actions.

5.3 Support for Development and Maintenance

- Requirements are captured as user stories and prioritized in a backlog; Git branches and pull requests support controlled changes.
- Every commit can trigger builds, unit tests, integration tests, security checks and policy validation.
- Container images are versioned and deployed to staging before controlled production release.
- Cloud environments and permissions can be defined declaratively through infrastructure-as-code.
- Service health, ingestion failures, anomaly-processing latency and automation outcomes are monitored continuously.
- New cloud services, pricing changes, detection improvements and policy updates can be delivered incrementally.
- Requirements are linked to user stories, commits, tests, deployments, recommendations and audit records.
- Remediation and application releases should support rollback when an action causes unexpected effects.

6. Expected Outcomes and Limitations

6.1 Expected Benefits

- Detect unexpected cloud expenditure earlier than periodic manual analysis.
- Improve resource utilization by identifying idle, oversized and under-utilized resources.
- Reduce unnecessary infrastructure cost through recommendations and controlled automation.
- Improve visibility by connecting billing, usage, configuration and deployment information.
- Reduce manual cloud-cost analysis and repetitive investigation work.
- Prevent some expensive configurations through IaC and CI/CD policy checks.
- Improve accountability through ownership metadata, approval workflows and audit trails.
- Support sustainable and efficient DevOps by making cost a continuous engineering consideration.

6.2 How Challenges Are Addressed

Challenge identified	How the solution addresses it
Unexpected cost spikes	Continuous ingestion, adaptive baselines and anomaly alerts provide earlier visibility.
Fragmented billing and technical data	Normalization and correlation connect costs with resources, utilization and deployment events.
False or noisy alerts	Severity, historical baselines, configurable thresholds and contextual signals reduce unnecessary alerts.
Difficult root-cause analysis	Anomaly evidence links resources to recent changes, deployments, utilization and owners.
Manual optimization	Recommendation and controlled remediation workflows reduce repetitive work.
Unsafe automation	RBAC, policy gates, approvals, restricted permissions and rollback controls limit risk.
Expensive IaC changes	Policy checks run during pull requests and CI/CD before infrastructure is deployed.
Weak traceability	Audit service records recommendations, approvals, actions and results with timestamps.

6.3 Risks, Implementation Challenges and Limitations

Risk / limitation	Mitigation
False positives or false negatives	Use historical validation, configurable sensitivity and periodic evaluation.
Changing workload patterns	Use adaptive baselines and allow known exceptions.
Incomplete or delayed billing data	Track freshness, retry imports and label incomplete periods.
Incorrect cost allocation	Introduce tagging policies, ownership metadata and data-quality checks.
Optimization affects performance	Show risk, require approval for sensitive actions, monitor results and support rollback.
Provider API limits or changes	Use modular adapters, retries/backoff and provider-specific tests.
Security compromise of automation credentials	Use least privilege, secrets management and audit logging.
Inaccurate savings estimates	Treat estimates as forecasts and validate actual savings after changes.
High telemetry volume	Use asynchronous processing, batching, retention controls and scalable storage.
Team inexperience	Start with limited scope, sandbox resources and clear documentation.

Overall limitations: the prototype may support only selected cloud providers and resource types, may depend on the quality and timeliness of billing/monitoring data, and may use rule-based or simplified anomaly detection rather than a production-grade predictive model. Real-world deployment would require security review, permission validation, cost-model validation and provider-specific testing.

6.4 Future Enhancements

- Multi-cloud support with a common cost model.
- Advanced time-series and machine-learning anomaly detection with explainable features.
- Predictive cost forecasting and budget simulation.
- Automated rightsizing based on utilization and performance constraints.
- Carbon-aware optimization and sustainability reporting.
- FinOps chargeback/showback dashboards.
- Policy-as-code libraries for common cloud governance and cost controls.
- Integration with ticketing, chat and incident-management platforms.
- Learning from approved/rejected recommendations and remediation outcomes.
- Advanced Kubernetes/container cost allocation by namespace, workload and team.

7. Conclusion

Cloud applications can experience unexpected cost increases because of resource leaks, configuration errors, excessive workloads and inefficient deployment practices. Manual analysis may identify these issues only after significant expenditure has already occurred. The proposed Cloud Cost Anomaly Detection and FinOps Automation Platform addresses this problem by combining continuous cost monitoring, anomaly detection, resource analysis, optimization recommendations, policy checks and controlled remediation.

The architecture connects cloud billing and usage data with monitoring, infrastructure-as-code, containers, CI/CD and audit trails. Dashboards and alerts improve visibility, while recommendation and approval workflows help teams reduce unnecessary infrastructure cost without introducing uncontrolled changes.

Applying software engineering principles such as modularity, scalability, maintainability, reliability, usability, security, testability and auditability makes the solution suitable for iterative development and long-term maintenance. Agile Scrum combined with DevOps is appropriate because the platform can evolve through short, tested increments while continuously integrating security, automation and operational feedback.

The expected result is earlier detection of unexpected expenditure, improved resource utilization, reduced unnecessary cost, stronger cloud-operation visibility and less manual FinOps analysis. Controlled automation and comprehensive audit trails ensure that cost optimization remains governed, traceable and aligned with application reliability and security.

8. References

- Sommerville, I., Software Engineering, 10th ed., Pearson, 2016.
- Pressman, R. S., Software Engineering: A Practitioner's Approach, 8th ed., McGraw-Hill, 2014.
- FinOps Foundation, FinOps Framework and FinOps Principles.
- Amazon Web Services, AWS Well-Architected Framework – Cost Optimization Pillar.
- Microsoft Azure, Azure Well-Architected Framework – Cost Optimization.
- Google Cloud, Google Cloud Architecture Framework – Cost Optimization.
- Kubernetes Documentation, Resource Management for Pods and Containers.
- Terraform Documentation, Infrastructure as Code and Terraform Workflows.
- Schwaber, K. and Sutherland, J., The Scrum Guide, 2020.
- OWASP Foundation, OWASP Application Security Verification Standard.

RUBRICS FOR EVALUATION

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains problem, objectives and scope with complete understanding.	Explains problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, roles and challenges with clear analysis.	Identifies most stakeholders with adequate analysis.	Identifies only a few stakeholders or challenges.	Analysis incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes existing system, strengths, weaknesses and gaps.	Good analysis with most strengths and weaknesses.	Limited analysis with partial issue identification.	Analysis largely missing or inappropriate.
Proposed Software Solution & Principles (15%)	Innovative, feasible solution applying appropriate principles.	Suitable solution with good application.	Basic solution with limited application.	Principles are inappropriate, poorly justified or absent.
Methodology / Framework (10%)	Selects the most suitable methodology with strong technical justification.	Suitable methodology with adequate justification.	Methodology selected but justification weak.	Methodology inappropriate or not discussed.

Assessment Summary

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology / Framework Justification (10%)	/5
TOTAL	/25

Student Name: N Reddy varaprasad reddy

Register Number: 192311045

Date: 07/10/2026

Signature: N Reddy